

Object-Oriented Programming

A. Class

Exercise 1. (Class)

Write a Python program to create a class `PersonDirectory` that stores a dictionary of IDs and names. Add a new record for Susan, modify John's name to John Smith, retrieve Alice's name, and print the final directory

Given:

```
class PersonDirectory:
    def __init__(self):
        self.records = {
            "111-34-3434": "John",
            "343-45-5455": "Alice"
        }
directory = PersonDirectory()
directory.records["234-56-9010"] = "Susan"
directory.records["111-34-3434"] = "John Smith"
print("Value for 343-45-5455:", directory.records["343-45-5455"])
print("Final Directory:", directory.records)
```



B. Objects

Exercise 1. (Creating objects)

Write a Python program to create a class Student and create three student objects. Add a new student object for Susan, modify Alice's name to John Smith, retrieve Bob's name, and print all student names

```
class Student:
    def __init__(self, name):
        self.name = name
student1 = Student("Alice")
student2 = Student("Bob")
student3 = Student("Susan")
student1.name = "John Smith"
print("Student2 name:", student2.name)
print("Final Students:", [student1.name, student2.name, student3.name])
```

Exercise 2. (Instance Variables)

Write a Python program to create a class Student with instance variables for ID and name. Add Susan, modify John's name, retrieve Alice's name, and print all students with their IDs

```
class Student:
    def __init__(self, id, name):
        self.id = id
        self.name = name
s1 = Student("111-34-3434", "John")
s2 = Student("343-45-5455", "Alice")
s3 = Student("234-56-9010", "Susan")
s1.name = "John Smith"
print("Student with ID 343-45-5455:", s2.name)
print("Final Students:", [(s.id, s.name) for s in [s1, s2, s3]])
```



Exercise 3. (Self-parameters)

Write a Python program to create a class Student that uses the self-parameter to show the student's name. Create a student object, change the name, and display it.

```
class Student:
    def __init__(self, name):
        self.name = name
    def show(self):
        print("Student Name:", self.name)
s1 = Student("Alice")
s1.name = "John Smith"
s1.show()
```

C. Inheritance

Exercise 1. (Single Inheritance)

Write a Python program to create a base class Person and a derived class Student. Add Susan, modify John's name, retrieve Alice's name, and print all students.

```
class Person:
    def __init__(self, name):
        self.name = name
class Student(Person):
    def __init__(self, id, name):
        super().__init__(name)
        self.id = id
s1 = Student("111-34-3434", "John")
s2 = Student("343-45-5455", "Alice")
s3 = Student("234-56-9010", "Susan")
s1.name = "John Smith"
print("Student with ID 343-45-5455:", s2.name)
print("Final Students:", [(s.id, s.name) for s in [s1, s2, s3]])
```





Exercise 2. (Multiple Inheritance)

Write a Python program to demonstrate multiple inheritance using classes Person and ID. Create a student class that inherits from both, then add Susan, modify John's name, retrieve Alice's name, and print all students.

```
class Person:
    def __init__(self, name):
        self.name = name

class ID:
    def __init__(self, id):
        self.id = id

class Student(Person, ID):
    def __init__(self, id, name):
        Person.__init__(self, name)
        ID.__init__(self, id)

s1 = Student("111-34-3434", "John")
s2 = Student("343-45-5455", "Alice")
s3 = Student("234-56-9010", "Susan")
s1.name = "John Smith"

print("Student with ID 343-45-5455:", s2.name)
print("Final Students:", [(s.id, s.name) for s
in [s1, s2, s3]])
```

```
CLASS Person:
    METHOD initialize(name):
        SET self.name TO name

CLASS ID:
    METHOD initialize(id):
        SET self.id TO id

CLASS Student INHERITS FROM Person AND ID:
    METHOD initialize(id, name):
        CALL Person.initialize(name)
        CALL ID.initialize(id)

CREATE s1 AS Student WITH id = "111-34-3434", name = "John"
CREATE s2 AS Student WITH id = "343-45-5455", name = "Alice"
CREATE s3 AS Student WITH id = "234-56-9010", name = "Susan"

UPDATE s1.name TO "John Smith"

PRINT "Student with ID 343-45-5455:", s2.name

FOR EACH student IN [s1, s2, s3]:
    ADD (student.id, student.name) TO final_list

PRINT "Final Students:", final_list
```





Exercise 3. (Multi-level Inheritance)

Write a Python program to demonstrate multi-level inheritance with classes Person, Student, and GraduateStudent. Add Susan, modify John's name, retrieve Alice's degree, and print all graduate students.

```
class Person:
    def __init__(self, name):
        self.name = name
class Student(Person):
    def __init__(self, id, name):
        super().__init__(name)
        self.id = id
class GraduateStudent(Student):
    def __init__(self, id, name, degree):
        super().__init__(id, name)
        self.degree = degree
s1 = GraduateStudent("111-34-3434", "John", "MSc")
s2 = GraduateStudent("343-45-5455", "Alice", "PhD")
s3 = GraduateStudent("234-56-9010", "Susan", "MBA")
s1.name = "John Smith"
print("Graduate Student with ID 343-45-5455:", s2.name, "-", s2.degree)
print("Final Students:", [(s.id, s.name, s.degree) for s in [s1, s2, s3]])
```



D. Encapsulation

Exercise 1. (Public)

Write a Python program to demonstrate public variables in a class Student. Add Susan, modify John's name, retrieve Alice's name, and print all students.

```
class Student:
    def __init__(self, id, name):
        self.id = id
        self.name = name
s1 = Student("111-34-3434", "John")
s2 = Student("343-45-5455", "Alice")
s3 = Student("234-56-9010", "Susan")
s1.name = "John Smith"
print("Public Name of s2:", s2.name)
print("Final Students:", [(s.id, s.name) for s in [s1, s2, s3]])
```

Exercise 2. (Private)

Write a Python program to demonstrate private variables in a class Student using getter and setter methods. Add Susan, modify John's name, retrieve Alice's name, and print all students.

```
class Student:
    def __init__(self, id, name):
        self.__id = id
        self.__name = name
    def get_name(self):
        return self.__name
    def set_name(self, new_name):
        self.__name = new_name
s1 = Student("111-34-3434", "John")
s2 = Student("343-45-5455", "Alice")
s3 = Student("234-56-9010", "Susan")
s1.set_name("John Smith")
print("Private Name of s2:", s2.get_name())
print("Final Students:", [s.get_name() for s in [s1, s2, s3]])
```





Exercise 3. (Protected)

Write a Python program to demonstrate protected variables in a class Student and access them through a subclass Graduate. Add Susan, modify John's name, retrieve Alice's name, and print all graduate students.

```
class Student:
    def __init__(self, id, name):
        self._id = id
        self._name = name
class Graduate(Student):
    def __init__(self, id, name, degree):
        super().__init__(id, name)
        self.degree = degree
s1 = Graduate("111-34-3434", "John", "MSc")
s2 = Graduate("343-45-5455", "Alice", "PhD")
s3 = Graduate("234-56-9010", "Susan", "MBA")
s1._name = "John Smith"
print("Protected Name of s2:", s2._name)
print("Final Students:", [(s._id, s._name, s.degree) for s in [s1, s2, s3]])
```

